Grenoble INP - Ensimag, UGA
Projet GL 2024-2025

Bilan de gestion d'équipe et de projet

Groupe 6 Equipe 34

BORNARD Maël, FOKOU DJONTU Manuella, GAUTIER Mattéo, JEULIN Matteo, MELLOT Arthur

Table des matières :

Table des matières : 1
1. Description critique de la méthode de développement et de l'organisation adoptée 2
a. Choix de la méthodologie 2
Outils utilisés 2
b. Organisation quotidienne 2
Horaires et communication 2
Réunions 2
c. Planification et gestion des délais 3
2. Répartition des tâches et rôles 3
3. Difficultés rencontrées et points positifs 4
a. Règles initiales non respectées 4
Tests et développement 4
Conséquences 4
b. Points positifs 5
4. Présentation de l'historique du projet 6
a. Contexte initial 6
b. Chronologie des activités 6
i. Langage Hello World 6
ii. Langage Sans Objet 6
iii. Langage Essentiel 7
iv. Extension OPTIM 7
v. Documentation 7
c. Analyse des activités et des temps passés 7
i. Répartition des activités 7
5. Conclusion 8
Annexe 1 : Planning prévisionnel 9
Annexe 2 : Planning réalisé 10

1. Description critique de la méthode de développement et de l'organisation adoptée

a. Choix de la méthodologie

Nous avons décidé d'adopter une approche agile pour mener à bien ce projet, en le découpant en plusieurs sprint et en ajustant notre planning lorsque nécessaire. Cette méthode nous a permis de rester flexibles face aux évolutions et aux imprévus, tout en favorisant une communication régulière et efficace au sein de l'équipe.

Outils utilisés

- **Trello** : Cet outil a été central pour la gestion des tâches. Nous avons organisé nos travaux en sprints en fonction des différents langages et rendus, en utilisant des colonnes pour visualiser les tâches à faire, celles en cours, et celles déjà réalisées.
- **Diagramme de Gantt** : Bien qu'il nous ait donné une vision d'ensemble des deadlines et jalons importants, cet outil a été peu utilisé au quotidien, éclipsé par Trello qui était plus adapté pour suivre les tâches en détail.

b. Organisation quotidienne

Horaires et communication

Nous avions convenu de travailler en présentiel à partir de 9h chaque jour. Cet engagement a été globalement respecté, facilitant une communication fluide et réactive au sein de l'équipe. Chaque membre était immédiatement disponible pour répondre à des questions sur du code qu'il a produit ou à des questions plus générales sur de la conception par exemple.

Réunions

Nous avions initialement prévu d'organiser des réunions quotidiennes de 10 minutes, inspirées du format "daily scrum". Cependant, la fréquence de ces réunions a varié selon les phases du projet et l'échange libre durant les séances de travail s'est imposé :

- **Travail sur des fonctionnalités distinctes** : Les réunions étaient plus fréquentes pour partager les avancées, discuter des difficultés rencontrées, et coordonner les efforts. Cela permettait de tenir informer l'ensemble des membres de l'équipe de l'avancée de nos tâches, de nos difficultés et c'était l'occasion de demander du renfort ou réfléchir sur des questions d'architecture.

- **Travail sur des fonctionnalités communes** : Les échanges étaient spontanés et réguliers tout au long de la journée, rendant les réunions formelles moins nécessaires.

c. Planification et gestion des délais

Le fait d'avoir prévu une marge d'une journée avant chaque rendu s'est avéré très bénéfique. Cela nous a permis de tester le code en profondeur et d'aborder les deadlines avec sérénité. Durant ces journée tampon une partie de l'équipe commençait les prochaines tâches tandis que l'autre partie corrigeait les derniers bugs, s'assurant d'avoir une couverture de tests optimale et vérifiant la qualité et propreté du code.

2. Répartition des tâches et rôles

Tout au long du projet, les tâches se sont réparties en fonction des appétences de chacun et des tâches disponibles à un instant donné.

Voici les principales tâches réalisé par chacun des membres du groupe

- **Mattéo Gautier** : Il s'est occupé du parseur pour l'entièreté du langage Decac ainsi que la représentation du programme avec objet par les différentes classes, puis a implémenté la génération de code pour les boucles « while » et l'ensemble du langage complet. Une partie importante de son travail a été dédiée au débogage, notamment en assembleur. Il a également écrit des scripts de tests pour la génération de code.
- **Arthur Mellot** : A d'abord travaillé sur la génération de code pour le langage sans objets (tout sauf Assign, If, While) et a aidé Mattéo Gautier sur la génération de code pour le langage essentiel. Ensuite, il s'est concentré sur l'extension, notamment la bibliographie, à la réflexion sur l'architecture et à son implémentation.
- **Mattéo Jeulin** : A commencé par rédiger les scripts de tests qui nous ont permis de tester notre code tout au long du projet. Il a ensuite implémenté les parties B et C pour les variables et la partie B pour le langage essentiel. Il a participé au débogage sur l'ensemble du projet et contribué à l'implémentation de l'extension.
- **Maël Bornard** : A démarré par l'implémentation du lexer, puis a travaillé sur la partie B pour les opérations ainsi que sur la génération de code pour les instructions conditionnelles (IfThenElse). Il a également réalisé du débogage, notamment sur la partie B, et écrit des scripts de tests.
- **Manuella Fokou Djontu** : A commencé par le lexer, puis a implémenté plusieurs fonctionnalités pour la partie B (IfThenElse, variables, conversions, cast, instanceof). Elle a aussi participé à l'implémentation de l'extension.

Tous les membres ont participé aux tests, garantissant la qualité globale du projet grâce à des points de vues et des idées de tests différents.

Mattéo Jeulin, en tant que chef de projet, a été à l'initiative de l'organisation des réunions, il coordonnait aussi les merges des différentes branches ainsi que la gestion globale du GilLab.

Une piste d'amélioration est une meilleure répartition des tâches notamment sur la fin du projet. En effet, la génération de code du langage essentielle à été majoritairement portée par une seule personne. Cela n'a pas posé de problème au départ du fait de la parallélisation des tâches. Néanmoins, une fois les autres tâches terminées, les autres membres pouvaient peu aider cette partie car il restait uniquement du débogage sur un code complexe théoriquement, pour lequel il faut du temps pour comprendre le principe.

3. Difficultés rencontrées et points positifs

a. Règles initiales non respectées

Tests et développement

Nous avions prévu que les tests seraient rédigés avant le développement et par une personne différente de celle qui implémentait la fonctionnalité. Cependant, cette règle n'a pas été strictement respectée :

- Les tests étaient souvent écrits par la même personne ayant développé la fonctionnalité. Ceci est dû au fait que la personne chargée du développement avait besoin de tests pour visualiser son avancement, il était donc compliqué de se coordonner pour avoir les tests au bon moment.
- La rédaction des tests avant le développement était partielle : nous rédigions quelques tests initiaux, puis complétions la base de tests après l'implémentation afin de couvrir l'ensemble des cas possibles. Une raison possible serait l'envie de chacun d'avancer le développement du projet plutôt que d'écrire des batteries de tests.

Conséquences

Cette organisation a parfois conduit à des incohérences entre les exigences du sujet et le code validé, du fait que l'implémentation et la validation soie faites par la même personne . Ces problèmes ont toutefois été détectés lors des phases de débogage, notamment lorsque le débogage était réalisé par une autre personne ou en pair programming.

Git

Malgré la volonté affirmée dès le début du projet de garder un historique git propre et clair, notre historique git n'est pas aussi propre que nous l'aurions voulu. Notre organisation en branches nous à parfois bloqué lors des merges notamment, où nous devions attendre la fin du développement d'une fonctionnalité avant de pouvoir merger puis repartir sur le développement de nouvelles fonctionnalités. Ce désagrément pourrait se régler facilement à l'avenir en organisant mieux les branches et en utilisant les fonctionnalités de git que nous étions hésitant à utiliser dans ce projet de peur de faire une mauvaise manipulation pouvant provoquer des erreurs de git.

b. Points positifs

Malgré ces difficultés, notre approche collaborative et les ajustements apportés en cours de route nous ont permis de respecter les deadlines et de livrer un projet de qualité.

Le fonctionnement agile, combiné à l'utilisation de Trello, à une communication efficace et à un travail en présentiel, a été un facteur clé de notre succès. Les ajustements effectués au fil du projet ont permis de pallier les limites de notre organisation initiale.

4. Présentation de l'historique du projet

a. Contexte initial

Les premiers jours du projet ont été consacrés à une compréhension globale du sujet afin que chaque membre de l'équipe connaisse les tenants et les aboutissants de chaque partie. Cette démarche a permis à tous les membres de comprendre l'utilité et l'intégration des différentes composantes, même celles sur lesquelles ils ne travaillaient pas directement.

Conformément à la méthodologie agile adoptée, nous avons organisé le projet en incréments, visant à produire régulièrement un compilateur fonctionnel sur des sous-parties du langage.

Nous vous invitons à vous référer aux annexes 1 et 2 ou aux fichiers Planning.planner et Realisation.planner pour avoir la temporalité respectivement des tâches prévues et des tâches réalisées.

b. Chronologie des activités

i. Langage Hello World

La première semaine a été dédiée à l'implémentation des trois étapes du compilateur pour le langage HelloWorld. Le jeudi 19 décembre 2024, nous disposions d'un compilateur fonctionnel, bien que quelques bugs mineurs aient été découverts par la suite. Cette phase initiale a servi de base pour le développement ultérieur.

ii. Langage Sans Objet

Pour le langage Sans Objet, nous avons choisi de paralléliser le développement en trois sous-parties relativement indépendantes :

- Variables
- Opérations
- Instructions conditionnelles

Chaque sous-partie a été développée et testée en parallèle, incluant les étapes B (vérification contextuelle) et C (génération de code).

Cette parallélisation a permis de gagner du temps par rapport à un développement linéaire. Cependant, elle a également engendré un surcoût lors de la fusion des fonctionnalités. Certaines interactions, comme les opérations sur les booléens qui nécessitent les instructions conditionnelles pour être testées, n'ont pas permis de tester l'ensemble du code produit avant le merge. Cette situation a conduit à deux jours supplémentaires de débogage, réalisé par une partie de l'équipe, tandis que l'autre commençait le développement du langage Essentiel.

iii. Langage Essentiel

Les parties B et C ont été développées en parallèle au début. Une fois la partie B achevée, trois jours ont été nécessaires pour finaliser et déboguer la génération de code. Durant ce temps, nous avons implémenté les fonctionnalités avancées telles que le cast et l'opérateur `instanceof`.

iv. Extension OPTIM

La bibliographie nécessaire à l'extension OPTIM a été réalisée parallèlement au développement du langage Essentiel. L'implémentation concrète a commencé à la fin de la semaine 3. La semaine supplémentaire allouée à cette extension a été déterminante pour produire une version aboutie, conforme à nos attentes qualitatives.

v. Documentation

Bien que la documentation ait été prévue tout au long du projet, nous avons concentré nos efforts sur l'implémentation. Elle a finalement été rédigée entre le 20 et le 23 janvier 2025, après la livraison du code. Ce choix a permis de bénéficier d'une vue d'ensemble, rendant la documentation plus pertinente et complète.

c. Analyse des activités et des temps passés

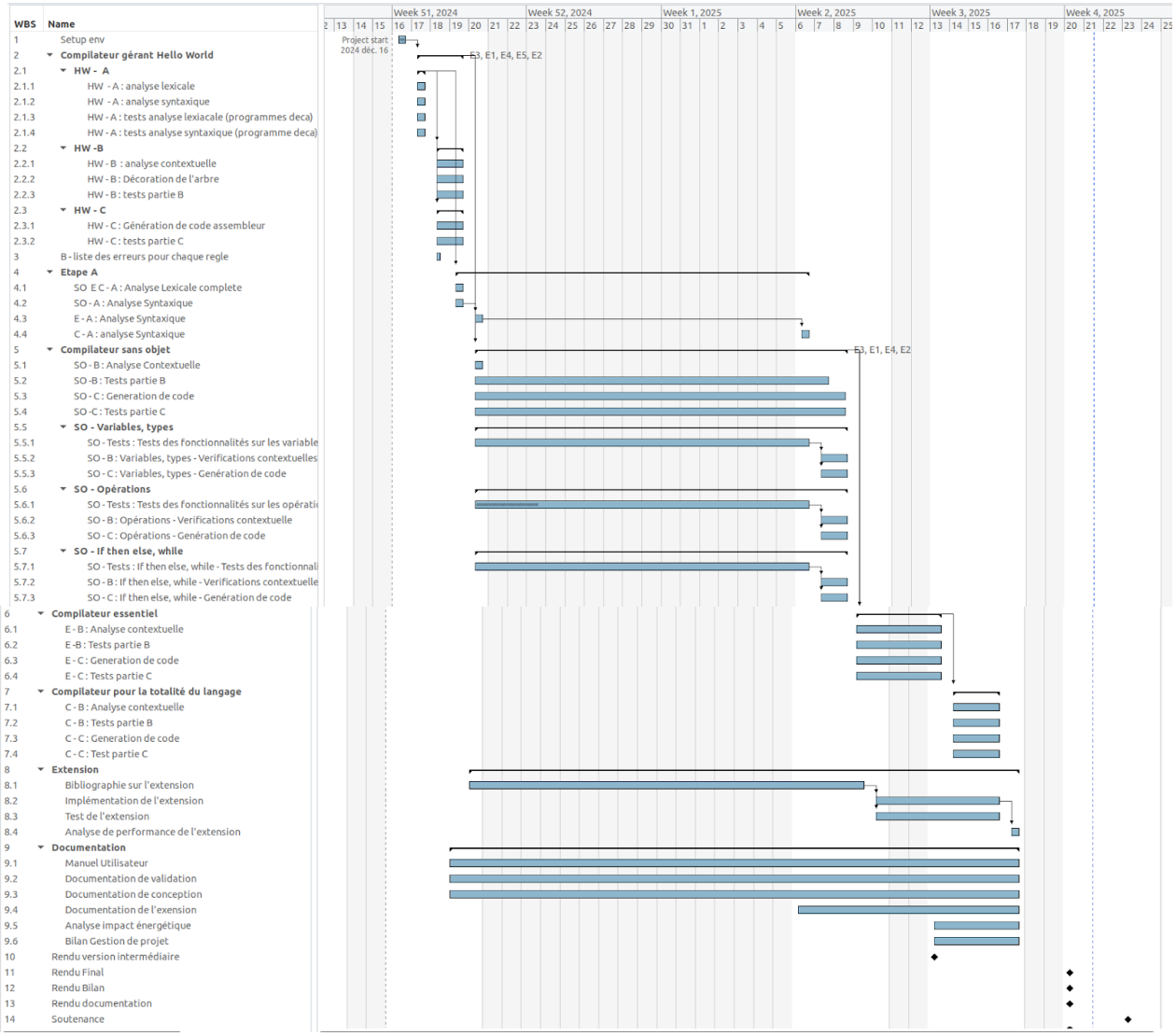
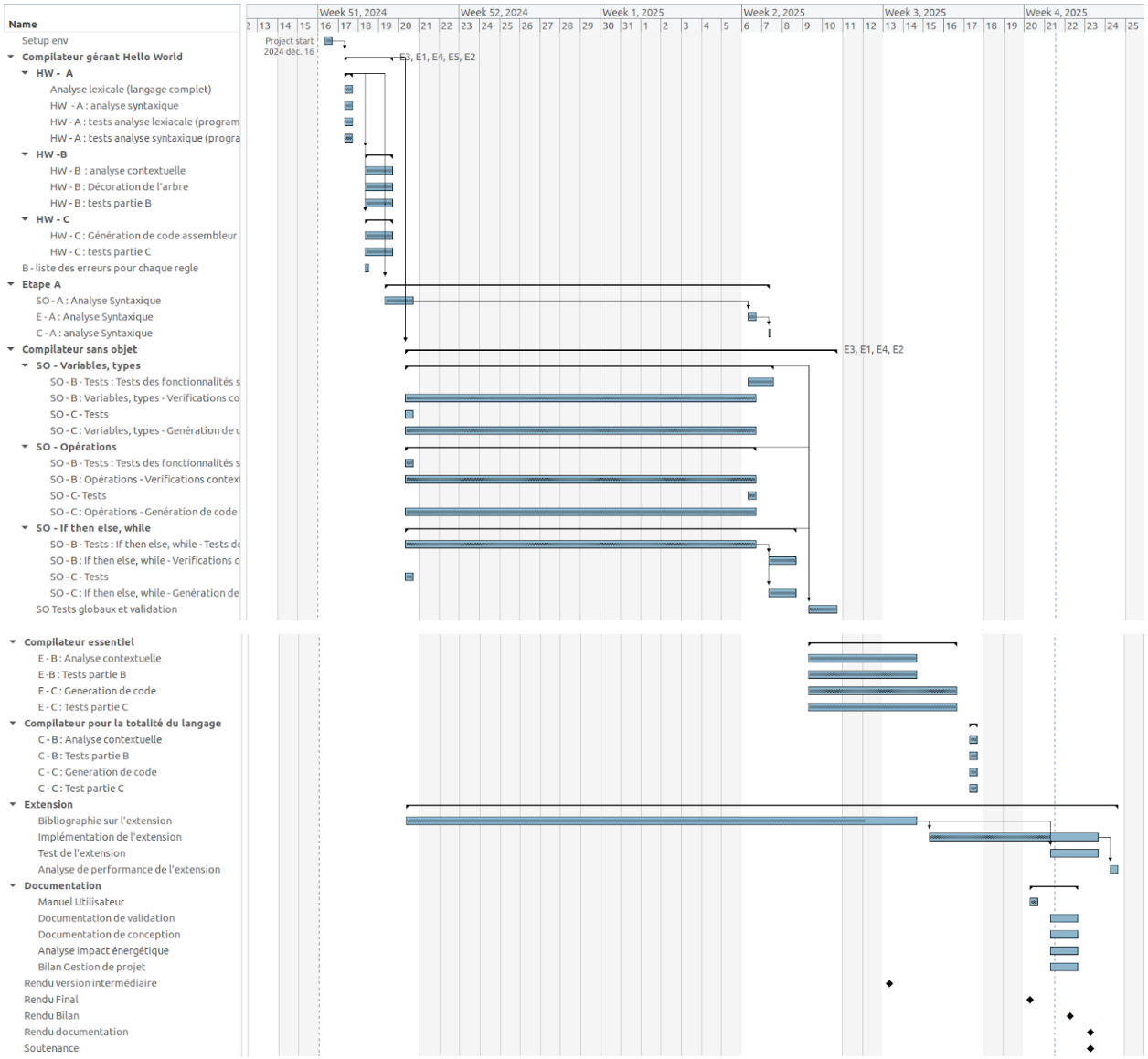
i. Répartition des activités

- **Analyse et conception** : 10% du temps total. Cette phase initiale a été essentielle pour poser les bases du projet.
- **Codage** : 55% du temps total. Le développement des différentes fonctionnalités a constitué l'essentiel de notre travail.
- **Validation et débogage** : 25% du temps total. Une part significative a été consacrée à l'intégration et à la résolution des problèmes.
- **Documentation** : 10% du temps total. Bien qu'elle ait été concentrée en fin de projet, elle a été réalisée avec soin.

5. Conclusion

En conclusion, ce projet a été mené à bien dans les temps impartis. Notre organisation nous a permis de répartir les tâches efficacement malgré quelques moments creux. Nous avons même eu de l'avance sur certaines étapes du développement. Nous avons réussi à bien gérer notre équipe et notre projet.

Annexe 1 : Planning prévisionnel



Annexe 2 : Planning réalisé